ASTANA IT UNIVERSITY

Department of Software Engineering | ShP-2216 Software Design Patterns

Assignment 06: Advanced In-Memory File System

Visitor Pattern & Double Dispatch Architecture

Student: Dias | **Weight:** 15% of RK2 | **Java 17 | Maven | JUnit 5**

GitHub Repository: https://github.com/Dias-AITU/sdp-assignment-06-visitor

---

1 Purpose and Architectural Design Choices

The objective of this assignment is to design an in-memory hierarchical file system that decouples structural object representation from arbitrary domain operations using the **Visitor Behavioral Pattern** and **Composite Structural Pattern**.

1.1 Double Dispatch Mechanics and Elimination of Type Inspection

Traditional object hierarchies often succumb to fragile `instanceof` cascades, reflection, or type-based branching when new features are added. This solution strictly utilizes **Double Dispatch**:

1. **First Dispatch (Element Selection):** The client invokes `element.accept(visitor)`. Runtime polymorphic dispatch dynamically routes the call to the concrete class (`FileNode` or `DirectoryNode`).
2. **Second Dispatch (Operation Selection):** Inside the concrete element, it invokes `visitor.visit(this)`. The compiler binds the overloaded visit method based on the exact compile-time type of `this`, executing either `visit(FileNode)` or `visit(DirectoryNode)`.

This pattern guarantees that behavioral selection is performed entirely through dynamic polymorphism with **zero runtime type checks**.

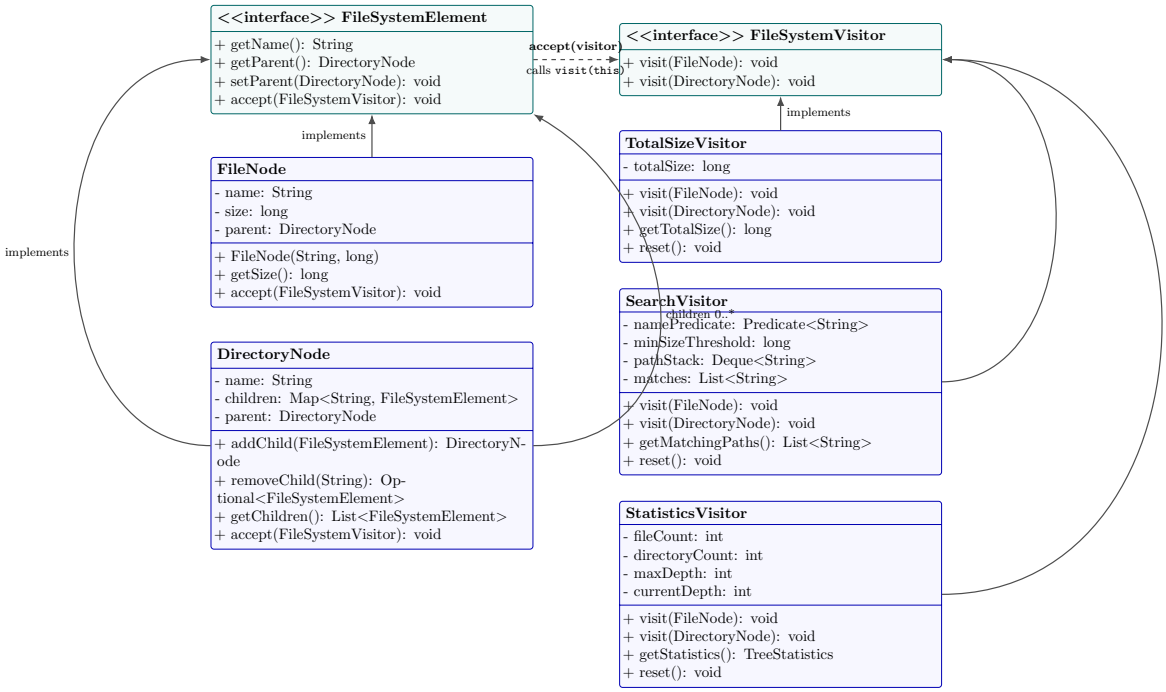
1.2 Tree Integrity and Invariant Enforcement (R6)

The file system preserves structural consistency across all mutations through synchronous assertions in constructors and the composite's `DirectoryNode.addChild()` method:

- **Non-Negative File Size:** Enforced in `FileNode(name, size)` via `InvalidFileSizeException`.
- **Duplicate Sibling Rejection:** `DirectoryNode` maintains children in a `LinkedHashMap<String, FileSystemElement>`. If an element with the same name already exists in the target directory, a `DuplicateNameException` is thrown.
- **Single-Parent Ownership:** Elements verify that `getParent() == null` before attachment. Attempting to add an element already attached elsewhere throws `MultipleParentException`.
- **Cycle Prevention:** When adding a child directory, the method iterates upward along the parent pointers of `this`. If any ancestor equals the incoming child, a `CycleDetectedException` is thrown.

2 UML Class Diagram

The diagram below illustrates the relationship between the composite element hierarchy and the visitor subsystem:


<<interface>> FileSystemElement
+ getName(): String
+ getParent(): DirectoryNode
+ setParent(DirectoryNode): void
+ accept(FileSystemVisitor): void
accept(visitor)
calls visit(this)
<<interface>> FileSystemVisitor
+ visit(FileNode): void
+ visit(DirectoryNode): void
implements
implements
TotalSizeVisitor
- totalSize: long
+ visit(FileNode): void
+ visit(DirectoryNode): void
+ getTotalSize(): long
+ reset(): void
FileNode
- name: String
- size: long
- parent: DirectoryNode
+ FileNode(String, long)
+ getSize(): long
+ accept(FileSystemVisitor): void
implements
children 0..*
SearchVisitor
- namePredicate: Predicate<String>
- minSizeThreshold: long
- pathStack: Deque<String>
- matches: List<String>
+ visit(FileNode): void
+ visit(DirectoryNode): void
+ getMatchingPaths(): List<String>
+ reset(): void
DirectoryNode
- name: String
- children: Map<String, FileSystemElement>
- parent: DirectoryNode
+ addChild(FileSystemElement): DirectoryNode
+ removeChild(String): Optional<FileSystemElement>
+ getChildren(): List<FileSystemElement>
+ accept(FileSystemVisitor): void
StatisticsVisitor
- fileCount: int
- directoryCount: int
- maxDepth: int
- currentDepth: int
+ visit(FileNode): void
+ visit(DirectoryNode): void
+ getStatistics(): TreeStatistics
+ reset(): void

3 Clean Code Justifications

The code adheres strictly to established Clean Code principles:

1. **Meaningful Naming (`SearchVisitor.buildRootRelativePath()`):**
   The method name explicitly communicates intent (constructing a '/'-separated path relative to the traversal root) without requiring inline explanatory comments.
2. **Single Responsibility Principle (`DirectoryNode` vs. `TotalSizeVisitor`):**
   `DirectoryNode` is solely responsible for maintaining child element relationships and enforcing tree integrity invariants, while calculation algorithms (size, search, statistics) are isolated inside independent visitor classes.
3. **Small, Focused Methods (`FileNode.accept()` and `DirectoryNode.accept()`):**
   The dispatch methods contain a single statement delegating to `visitor.visit(this)`, keeping cyclomatic complexity at exactly 1 and maximizing testability.
4. **Encapsulation of Internal State (`DirectoryNode.getChildren()`):**
   The internal mutable child map is completely shielded from external modification by returning `Collections.unmodifiableList(new ArrayList<>(children.values()))`.
5. **Elimination of Type-Checking Chains (`FileSystemElement.accept()`):**
   No `instanceof`, reflection, or conditional branching exists to differentiate between files and directories; method routing is handled entirely by double dispatch polymorphism.

4 Architectural and Extension Trade-Offs

4.1 Traversal Ownership: Visitor-Owned vs. Element-Owned (R7)

We selected **Visitor-Driven Traversal**. In this architecture, composite elements expose their unmodifiable children, and each visitor dictates how and when to traverse them.

- **Advantage 1 (Context Tracking):** Operations such as `SearchVisitor` and `StatisticsVisitor` require contextual state (a path stack for root-relative paths and a current depth counter). The visitor pushes state before visiting child nodes and pops state upon return. If traversal were embedded in `DirectoryNode`, this context would have to be passed awkwardly through element method arguments.
- **Advantage 2 (Traversal Control):** Specific visitors can prune branches (e.g., skip hidden or permission-restricted folders) without polluting the composite element's structure.

4.2 Extension Boundary Trade-Offs (The Visitor Dilemma)

The Visitor pattern introduces an explicit architectural trade-off:

- **Adding Operations (Extremely Easy):** Adding a new operation, such as `StatisticsVisitor` (R8) or a hypothetical `ChecksumVisitor`, is achieved by creating a new class implementing `FileSystemVisitor`. Zero lines of code in `FileNode`, `DirectoryNode`, or existing visitors are modified. The Open-Closed Principle is strictly upheld.
- **Adding Element Types (Hard):** If a new element type is introduced (e.g., `SymlinkNode`), the `FileSystemVisitor` interface must add `visit(SymlinkNode)`, which immediately breaks and requires updates across all existing visitor implementations (`TotalSizeVisitor`, `SearchVisitor`, and `StatisticsVisitor`).

5 Verification Suite & Execution Output

5.1 Automated JUnit 5 Test Suite Results

All 8 mandatory verification targets executed cleanly using `mvn test` with zero failures:

Check	Test Method Name	Verification Target	Result
T1	testT1_DoubleDispatchReaches...	Pure double dispatch routing	**PASSED**
T2	testT2_SizeAccumulationAcross...	Multi-level subtree byte size	**PASSED**
T3	testT3_SearchCombinesName...	Dual-filter search constraints	**PASSED**
T4	testT4_SearchProducesDeterministic...	Lexicographically sorted '/' paths	**PASSED**
T5	testT5_StatisticsCorrectForNesting...	File/dir count & max tree depth	**PASSED**
T6	testT6_EmptyStructuresAndNoMatch...	Empty directories & search misses	**PASSED**
T7	testT7_RepeatExecutionsDoNotLeak...	State isolation & visitor reuse	**PASSED**
T8	testT8_InvalidTreeMutationsRejected...	Tree integrity invariants (R6)	**PASSED**

5.2 Deterministic Program Output (kz.aitu.shp.filesystem.Main)

```
================================================================================
  OPERATION 1: TotalSizeVisitor (R3)
--------------------------------------------------------------------------------
Total Subtree Byte Size: 141180 bytes (137.87 KB)

--------------------------------------------------------------------------------
  OPERATION 2: SearchVisitor (R4)
--------------------------------------------------------------------------------
Query A: Filename contains '.java' AND size >= 3000 bytes
  [1] src/main/java/Application.java
  [2] src/main/java/OrderService.java
  [3] src/test/java/OrderServiceTest.java

Query B: All files with size >= 5000 bytes
  [1] docs/architecture-spec.json
  [2] docs/system-design.pdf
  [3] target/ecommerce-core-1.0.jar

--------------------------------------------------------------------------------
  OPERATION 3: StatisticsVisitor (R5 & R8)
--------------------------------------------------------------------------------
Hierarchy Statistics Summary:
  * Total Files Visited:       14
  * Total Directories Visited: 10 (including root)
  * Maximum Tree Depth:        4 (root depth = 0)

--------------------------------------------------------------------------------
  TREE INTEGRITY ENFORCEMENT (R6)
--------------------------------------------------------------------------------
1. Duplicate sibling prevention: PASSED -> Caught DuplicateNameException
2. Negative size prevention:     PASSED -> Caught InvalidFileSizeException
3. Cycle prevention:             PASSED -> Caught CycleDetectedException
================================================================================
```